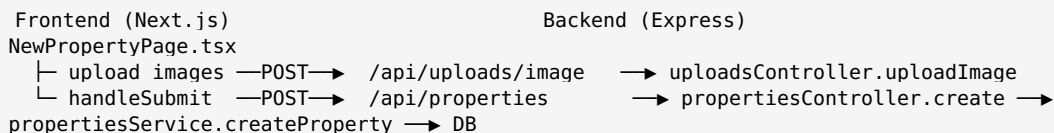


Documentation - Ajout de propriété

Documentation — Ajout d'une propriété

1. Architecture générale



Le formulaire fait **deux types d'appels** : d'abord un upload par image (une requête par fichier), puis un seul `POST /api/properties` avec toutes les URLs d'images déjà obtenues.

2. Compression d'image avant upload

```
async function compressImage(file, maxWidth = 1920, maxHeight = 1920, quality = 0.7) {
  if (!file.type.startsWith("image/")) return file;
  if (file.size < 300 000) return file; // déjà léger, pas la peine
  const bitmap = await createImageBitmap(file);
  // redimensionne si trop grand, dessine sur un canvas, convertit en webp
  ...
}
```

Avant d'envoyer une image au serveur, elle est redimensionnée (max 1920px) et convertie en **webp** côté navigateur, via `canvas.toBlob`. Ça réduit la taille des fichiers envoyés et accélère l'upload. Si l'image fait déjà moins de 300 Ko, on ne la retouche pas.

3. Les handlers de fichiers (`handleCoverChange`, `handlePicturesChange`, `handleHostPictureChange`)

```
const handleCoverChange = useCallback(async (e) => {
  const file = e.target.files?.[0];
  if (!file) return;
  const compressed = await compressImage(file);
  setCoverFile(compressed);
  setCoverPreview((prev) => {
    if (prev) URL.revokeObjectURL(prev);
    return URL.createObjectURL(compressed);
  });
}, []);
```

Ce même schéma est répété trois fois (couverture, galerie, photo de l'hôte) :

1. Récupère le/les fichier(s) choisis dans l' `<input type="file">`.
2. Comprime l'image (voir section 2).
3. Stocke le fichier compressé dans le state (`coverFile`, `pictureFiles`, `hostPictureFile`) — c'est ce fichier qui sera envoyé au serveur au submit.
4. Génère un aperçu local avec `URL.createObjectURL()` (affichage instantané, sans attendre le serveur), en libérant l'ancien aperçu avec `URL.revokeObjectURL()` pour éviter les fuites mémoire.

`handlePicturesChange` fait la même chose mais pour plusieurs fichiers à la fois (`multiple`), et **ajoute** aux images déjà présentes au lieu de remplacer.

4. `uploadImage` — envoi d'une image au serveur

```
const uploadImage = async (file) => {
  const formData = new FormData();
  formData.append("file", file);
  const res = await api.post("/api/uploads/image", formData, {
    headers: { "Content-Type": "multipart/form-data" },
  });
```

```
});
return res.data.url;
};
```

Envoie le fichier dans un `FormData` (format obligatoire pour l'upload de fichiers) vers `/api/uploads/image`, et récupère l'URL publique renvoyée par le serveur (ex : `/uploads/173920-a1b2c3d4.webp`). C'est cette URL qui sera stockée en base, pas le fichier lui-même.

5. `handleSubmit` — le cœur de la création

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setError(null);

  if (!title.trim()) return setError("Le titre est obligatoire.");
  if (!pricePerNight || isNaN(Number(pricePerNight)))
    return setError("Le prix par nuit est obligatoire.");

  setSubmitting(true);

  try {
    const [coverUrl, pictureUrls, hostPictureUrl] = await Promise.all([
      coverFile ? uploadImage(coverFile) : Promise.resolve(undefined),
      Promise.all(pictureFiles.map((file) => uploadImage(file))),
      hostPictureFile ? uploadImage(hostPictureFile) : Promise.resolve(undefined),
    ]);

    const payload = {
      title: title.trim(),
      description: description.trim() || undefined,
      cover: coverUrl,
      location: `${postalCode} ${location}`.trim() || undefined,
      price per night: Number(pricePerNight),
      host: hostName.trim() ? { name: hostName.trim(), picture: hostPictureUrl } : undefined,
      pictures: pictureUrls,
      equipments: selectedEquipments,
      tags: selectedTags,
    };

    const res = await api.post("/api/properties", payload);
    router.push(`/properties/${res.data.id}`);
  } catch (err) {
    setError(err?.response?.data?.error || "Une erreur est survenue.");
  } finally {
    setSubmitting(false);
  }
};
```

Étapes, dans l'ordre :

1. Validation front minimale : titre obligatoire, prix obligatoire et numérique.
2. **Point clé de perf** : les 3 uploads (couverture, galerie, photo hôte) partent **en parallèle** avec `Promise.all`, au lieu d'attendre l'un après l'autre. Ça évite d'additionner les temps de latence de chaque upload.
3. Une fois toutes les URLs récupérées, construction du `payload` : les champs vides deviennent `undefined` (pour ne pas les envoyer inutilement), le prix est converti en nombre, et `host` n'est inclus que si un nom d'hôte a été saisi.
4. Envoi du `payload` en un seul `POST /api/properties`.
5. Si succès : redirection vers la page de la propriété créée (`/properties/{id}`).
6. Si échec : affichage du message d'erreur renvoyé par le backend.

6. Backend — upload d'image (`uploadsController.uploadImage`)

- Utilise **multer** pour gérer le `multipart/form-data` : stocke le fichier sur disque (`public/uploads/`), avec un nom généré (timestamp + suffixe aléatoire) pour éviter les collisions.
- `fileFilter` rejette tout ce qui n'est pas une image (vérifie le `mimetype`).
- Limite de taille : 10 Mo par fichier.
- Renvoie `{ url, filename, size, mimetype }` — c'est cette `url` que le frontend récupère et stocke dans le `payload`.

7. Backend — création de la propriété (`propertiesController.create` → `propertiesService.createProperty`)

1. Vérifie que `title` est présent (sinon `400`).
 2. Le prix est validé : si absent ou invalide, valeur par défaut de `80`.
 3. **Gestion de l'hôte** (`ensureHost`) : si un `host_id` existe déjà, on le réutilise ; sinon, si un objet `host` : `{ name, picture }` est fourni, on cherche un utilisateur existant avec le même nom/photo, sinon on en crée un nouveau (`role: 'owner'`).
 4. Génère un `slug` unique à partir du titre (ex : "Appartement Cosy" → `appartement-cosy`, ou `appartement-cosy-2` si déjà pris) — utile pour des URLs propres.
 5. Insère la propriété en base, puis insère séparément les lignes liées : `property_pictures`, `property equipments`, `property_tags` (une ligne par élément, avec `INSERT OR IGNORE` pour éviter les doublons).
 6. Renvoie les détails complets de la propriété créée (avec ses photos, équipements, tags et infos de l'hôte).
-

8. Sécurité des routes

```
router.post('/properties', requireRole(['owner', 'admin']), properties.create);
router.post('/uploads/image', requireRole(['owner', 'admin']), uploads.uploadImage);
```

Seuls les utilisateurs avec le rôle `owner` ou `admin` peuvent créer une propriété ou uploader une image — un `client` ne peut pas publier de logement. Le contrôle se fait via le middleware `requireRole`, avant même d'atteindre le controller.

9. Résumé express

- **Upload d'images** : compression côté client (webp, redimensionnement) → un `POST /api/uploads/image` par fichier → l'URL renvoyée est stockée en state.
- **Soumission** : validation front → tous les uploads en parallèle (`Promise.all`) → construction du payload → un seul `POST /api/properties` → redirection.
- **Backend** : valide le titre/prix → résout ou crée l'hôte → génère un slug unique → insère propriété + photos + équipements + tags → renvoie l'objet complet.
- **Sécurité** : routes de création réservées aux rôles `owner` / `admin`.